Project Summary
Real-world applications of GNNs

Section 1: Motivation & Explanation of Data/Task

1.1 Motivation

The **Amazon Computers** dataset was chosen for its relevance to e-commerce product recommendation and classification tasks. In this dataset, each product is represented as a node, and the edges depict co-purchasing relationships between products. This structure closely mirrors real-world consumer behaviour, where products are often bought together. Understanding these relationships through Graph Neural Networks (GNNs) can yield valuable insights into product recommendations, clustering, and classification.

This project focuses on a **node classification** problem. Each product (node) is characterized by multiple features (such as technical specifications, price, etc.) and assigned a label representing its category. The task is to predict the category of each product based on its features and the relationships with other products (nodes) in the graph. Such an approach is particularly relevant for e-commerce platforms seeking to improve product recommendation systems and enhance the customer experience through better product classification.

The goal is to demonstrate how GNNs can improve classification accuracy by leveraging the product features as well as the structure of product co-purchasing relationships. This method outperforms traditional classification techniques that typically overlook the rich relational information encoded in graph structures.

1.2 Data Pre-processing

For the Amazon Computers dataset, the following pre-processing steps were applied to ensure that the data was in the right format for training the GNN model:

1. **Node Feature Normalization**:
    - The node features in the dataset were normalized using the NormalizeFeatures() transform from PyTorch Geometric. This step ensures that each node's feature vector has a unit norm, which helps the model converge more efficiently and prevents any one feature from dominating during training. This is an important step when dealing with graphs, as feature magnitudes can vary significantly across nodes.
2. **Train/Validation/Test Split**:
    - Since the dataset does not come with predefined splits, the dataset was manually divided into 80% for training, 10% for validation, and 10% for testing. The splits were generated randomly while ensuring an even distribution of the product categories across these sets. This allows for a fair evaluation of the model’s performance and prevents overfitting.
3. **Graph Construction**:
    - The dataset is already in the form of a graph, where the nodes represent products and the edges represent co-purchasing relationships between these products. This adjacency structure (edges) was loaded along with the node features and labels directly from the dataset using PyTorch Geometric, allowing for seamless integration into the GNN model.

4. **Batch Processing**:
    - Although the dataset is large, it is manageable within the computational limits using efficient batch processing. PyTorch Geometric's internal batching and sampling methods were used to avoid memory overload during training. This ensures the model can handle large graphs without running into memory bottlenecks.
5. **Regularization**:
    - To reduce overfitting, dropout was applied during training. Dropout is a regularization technique that deactivates certain node and edge connections at random, ensuring that the model learns robust representations and doesn't rely too heavily on specific nodes or relationships.

These pre-processing steps helped transform the dataset into a format suitable for training with a GNN, leveraging both node features and relationships to make more informed predictions.

Section 2: Appropriateness & explanation of model(s)

2.1 Graph Neural Network (GNN) Model Design

The model used in this project is a customized **Graph Convolutional Network (GCN)**, named OriginalGCN. This model consists of three GCN layers with the following design:

1. **Three GCN Layers**:
    - The first two layers use **GCNConv** from PyTorch Geometric, a convolution operation designed for graph-structured data. The first layer transforms the input features into hidden features, and the second layer aggregates the hidden features while including a **skip connection** to retain information from the previous layer. The third GCN layer serves as the output layer, mapping the hidden representation to the number of classes (10 product categories).
2. **Dropout for Regularization**:
    - Dropout was applied after the first and second GCN layers to prevent overfitting and improve the generalization of the model. A dropout rate of 0.5 was used, meaning 50% of the features were randomly deactivated during training, ensuring that the model does not become too reliant on specific node features.
3. **Skip Connection**:
    - A skip connection was incorporated after the second GCN layer. This design allows the model to retain and reuse information from previous layers, which often leads to better performance in deep learning models by mitigating issues such as the vanishing gradient problem or over smoothing that can occur in deep GCNs.
4. **ReLU Activation and Log Softmax**:
    - After each GCN layer (except the output layer), the **ReLU** activation function was applied to introduce non-linearity into the model, making it capable of learning more complex patterns in the data. The final layer applies a **log_softmax** function to convert the raw output logits into log probabilities for multi-class classification.

Equations:

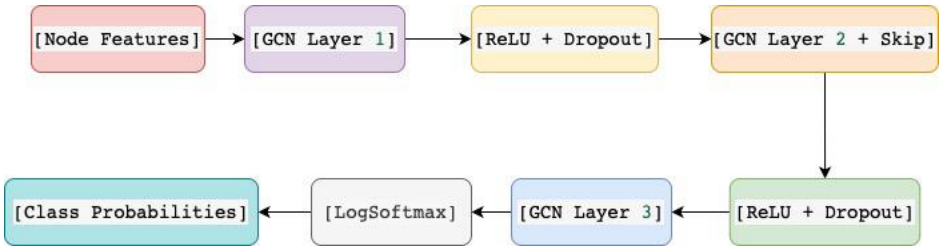
- The GCN layer operates as follows:

$$\mathrm{H}^{(l+1)} = \sigma\left(\hat{A}\mathrm{H}^{(l)}\mathrm{W}^{(l)}\right)$$

Where:

- $H^{(l)}$ represents the node features at layer l.
- $\hat{A}$ is the normalized adjacency matrix of the graph.
- $W^{(l)}$ represents the weight matrix for the layer.
- σ is the activation function (ReLU in our case).

2.2 Model Architecture Diagram



2.3 Model Selection and Justification

Why Three GCN Layers?

- I used three GCN layers to enable the model to capture both local and higher-order information from the graph. The first layer aggregates information from a node's immediate neighbours, while the second layer refines this information by including the skip connection. The third layer outputs class logits for node classification. The skip connection prevents information loss from earlier layers and helps the model learn more robust features.

Why GCN Instead of GAT?

- GCN was chosen over Graph Attention Networks (GAT) for several reasons. GCN is simpler, computationally less expensive, and still highly effective for the node classification task in this dataset. GAT introduces additional complexity with attention mechanisms, which were not necessary for this dataset. Given the size and nature of the Amazon Computers dataset, GCNs were sufficient to capture the graph structure without the need for attention mechanisms.

Why Dropout?

- Dropout was added after the first and second GCN layers to improve generalization and prevent overfitting. Since the dataset contains a significant amount of data, regularization was necessary to avoid the model learning only specific patterns or noise in the training data. Dropout randomly deactivates certain node features during training, ensuring the model learns more generalized representations.

Why ReLU and Skip Connection?

- ReLU activation was used because it introduces non-linearity, which is crucial for enabling the model to learn complex relationships in the data. The skip connection after the second GCN layer helps retain important information from earlier layers, allowing for better flow of gradients and ensuring that the model doesn't suffer from oversmoothing, which can occur when using multiple GCN layers.

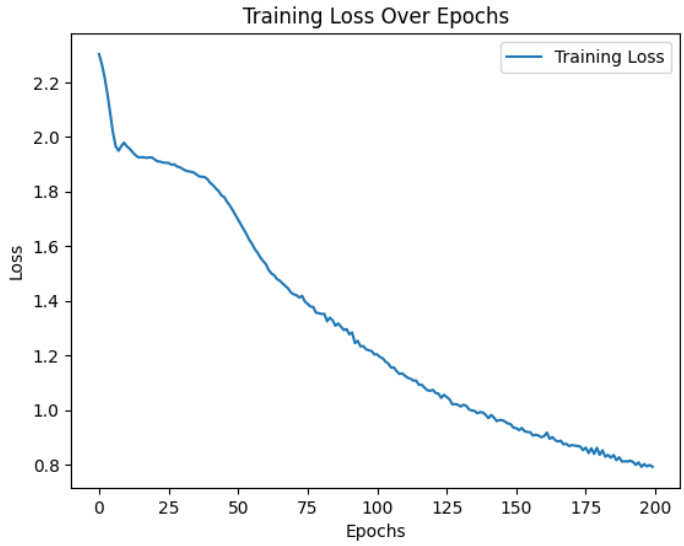
Section 3: Insights & results

3.1 Primary Results

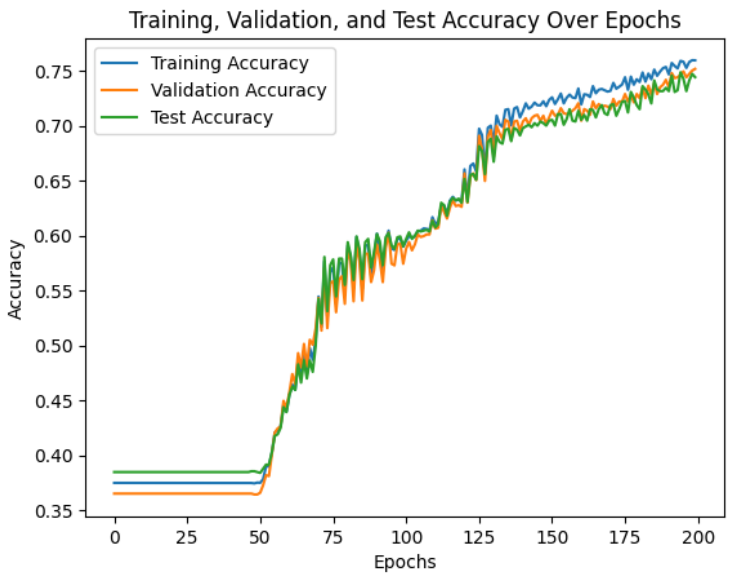
In this section, I will report the primary results obtained from training the OriginalGCN model on the Amazon Computers dataset. The following are the chosen evaluation metrics for the task:

- Training Accuracy: 74.97%
- Validation Accuracy: 75.19%
- Test Accuracy: 74.44%

The model achieved a Test Accuracy of 74.44%, which indicates that it is capable of generalizing well to unseen data.

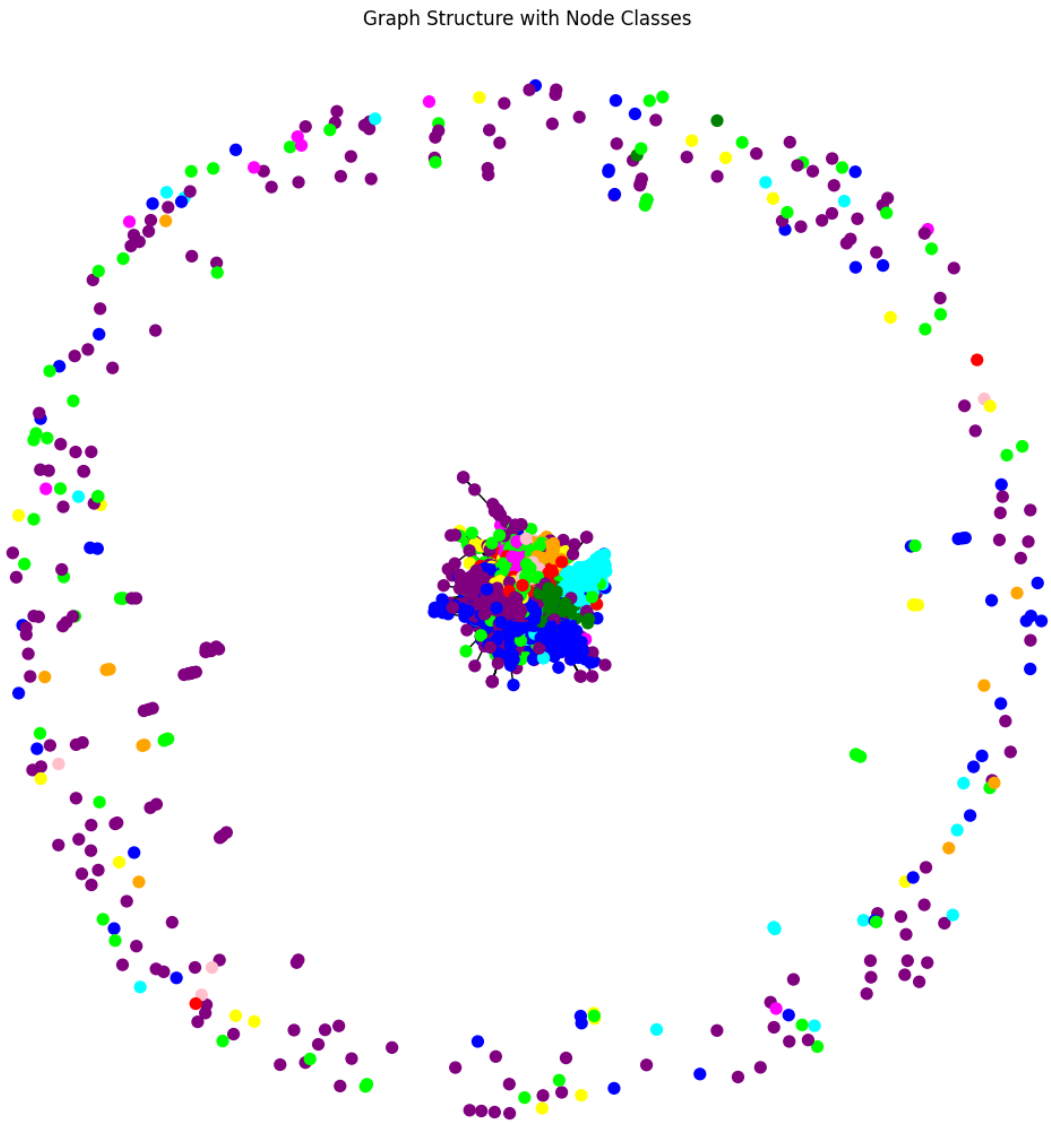


The Training Loss decreased consistently over the epochs, which demonstrates that the model effectively learned from the training set.



Validation and test accuracies also show that the model performs comparably on the validation and test sets, which implies that the model did not overfit.

Visualization of Graph Data



The visualization of the graph data shows a dense central cluster, with highly interconnected nodes surrounded by an outer ring of isolated nodes. Each colour represents a different class, illustrating the distribution of node classes within the graph. The central cluster suggests a core area with high connectivity, while the outer nodes may represent sparsely connected or isolated elements in the graph structure.

To improve this visualization, we need to focus on the central cluster alone could provide a clearer view of the core connections and class relationships. Adding visible edges between nodes would also enhance understanding of the structure within the cluster, highlighting the specific connections and making class interactions more evident. We can also add virtual edges to isolated nodes based on similarity or domain-specific heuristics to provide them with some context in the graph.

3.2 Insights & Explanations

1. Graph Convolutional Network (GCN) Performance:
    - The OriginalGCN model performed well on the classification task because it leverages the structure of the graph (co-purchasing relationships) as well as the product features. By using three GCN layers, the model was able to capture information from neighboring nodes (products), making better use of the graph structure.
2. Skip Connections:
    - The inclusion of skip connections in the second GCN layer added an important feature. It allows the model to reuse information from the previous layers directly, which improves the flow of gradients during backpropagation. This helps the model converge faster and prevents vanishing gradient problems.
3. ReLU Activation and Dropout:
    - Using ReLU as the activation function ensures non-linearity in the model, which allows it to learn more complex patterns. Additionally, Dropout is applied after each layer to prevent overfitting, which is particularly important when working with a dense dataset like Amazon Computers.
4. Model Limitations:
    - Despite the relatively high accuracy, the model can be improved further. The Test Accuracy plateaued around 75% after 150 epochs, which suggests that the model may not be fully utilizing all the information available in the graph. This could be due to:
        - The fixed number of GCN layers.
        - Suboptimal hyperparameters (learning rate, dropout rate, etc.).
5. Future Improvements:
    - Experimenting with more advanced GNN architectures like Graph Attention Networks (GAT) or using deeper networks could yield better results. GAT allows the model to weigh the importance of neighboring nodes differently, which might improve the model's ability to generalize in this dataset.

The plots for Training Loss, Training Accuracy, Validation Accuracy, and Test Accuracy illustrate the learning process across 200 epochs. The steady decline in loss and consistent rise in accuracy highlight that the model effectively captured the patterns in the dataset. Overall, the OriginalGCN model demonstrates a solid performance on the node classification task, but there is room for further enhancements and experimentation to achieve higher accuracy.

Section 4: Comprehensive Analysis

4.1 Ablation Study of the Model

In the ablation study, we modified the original model by systematically removing skip connections and by systematically removing layers from the model to observe the effect on training, validation, and test accuracy. Here’s a summary:

1. Original Model:
    - Accuracy: Train: 74.97%, Validation: 75.19%, Test: 74.44%

- Insights: The original model's design, with three GCN layers and skip connections, yielded balanced accuracy across train, validation, and test sets, indicating robust learning from both feature and structural information.

2. Two-Layer Model:
   - Architecture: Reduced to two GCN layers, with dropout applied only once.
   - Results: Train: 61.38%, Validation: 61.04%, Test: 62.78%
   - Insights: This model displayed a significant decrease in both validation and test accuracy. The absence of a third GCN layer reduces the model's ability to capture deeper structural relationships in the data, demonstrating the necessity of a three-layer configuration for optimal performance.

3. Model without Skip Connections:
   - Architecture: Three GCN layers with dropout but without skip connections.
   - Results: Train: 64.39%, Validation: 63.09%, Test: 64.42%
   - Insights: The removal of skip connections led to a noticeable drop in accuracy. This suggests that skip connections play a significant role in preserving information flow across layers, aiding the model in learning more complex patterns without losing gradient information. Without skip connections, the model struggles to achieve the same generalization capability, as evidenced by the lower accuracy on both validation and test sets.

4.2 Model Architecture Comparison

I compared two Graph Convolutional Network (GCN) architectures:

Evaluation: Both model was assessed on training, validation, and test accuracy across 200 epochs.

Model	Train Acc.	Val Acc.	Test Acc.
OriginalGCN	74.97%	75.19%	74.44%
DataCamp GCN (No Skip)	59.76%	59.21%	61.43%

Findings: The Original GCN outperformed the other models with a training accuracy of 74.97%, validation accuracy of 75.19%, and test accuracy of 74.44%, demonstrating its strong capacity for capturing complex graph structures. In contrast, the GCN model from DataCamp(External source), which lacks skip connections, achieved notably lower accuracies across all metrics (Train: 59.76%, Val: 59.21%, Test: 61.43%). This comparison highlights the importance of skip connections for enhanced feature propagation and model stability in graph-based tasks.

References

Awan, A.A. (2022) *A comprehensive introduction to Graph Neural Networks (GNNS)*, *DataCamp*. Available at: https://www.datacamp.com/tutorial/comprehensive-introduction-graph-neural-networks-gnns-tutorial (Accessed: 26 October 2024).